INCREMENTAL SDLC DOCUMENT

Electricity Generation Forecasting Project

Prepared from the supplied Python notebook: *electricity_bill_prediction.ipynb*

SDLC Model	Incremental Model
Primary Goal	Forecast electricity generation across monthly, daily, and hourly time scales
Core Technology	Python, Pandas, NumPy, PyTorch, Scikit-learn, Matplotlib
Notebook	electricity_bill_prediction.ipynb

Scope note: The notebook filename says “bill prediction,” but the described implementation forecasts electricity generation in GWh. It does not currently calculate an individual consumer's bill in rupees. This document reflects that implemented scope.

1. Introduction

This project applies deep learning to historical electricity data to forecast future electricity-generation values. The notebook uses a Python workflow for data preparation, time-series sequence construction, Transformer model training, and forecasts at different temporal resolutions. The Incremental SDLC model is appropriate because the workflow can be developed and validated in independent stages.

2. Project Objectives

- Load and preprocess historical electricity-generation data.
- Aggregate state-level records into a chronological time series.
- Convert observations into supervised learning sequences using lookback windows and forecast horizons.
- Train a Transformer neural network to learn temporal patterns.
- Generate monthly, daily, and hourly forecasts.
- Validate pipeline behavior and evaluate performance using held-out data and suitable error metrics.

3. SDLC Model: Incremental Model

The Incremental Model divides development into smaller releases. Each increment delivers a functional component, is tested, and becomes the foundation for the next increment. The increments for this project are data preparation, time-series dataset construction, Transformer development, multi-frequency forecasting, and testing/evaluation.

Increment	Activities	Deliverable
1. Data preparation	Load, filter, clean, aggregate, and sort records.	Clean chronological time series.
2. Sequence construction	Build lookback windows and target sequences.	Valid input-target tensors.
3. Transformer model	Define model layers and train on sequences.	Model produces numerical forecasts.
4. Forecasting	Run monthly, daily, and hourly workflows.	Timestamped forecast outputs.
5. Testing/evaluation	Test data flow, shapes, timestamps, and errors.	Verified behavior and evaluation results.

4. Increment-wise Development

Increment 1: Data Collection and Preprocessing

Objective: Prepare raw electricity records for modeling. The workflow loads CSV data with Pandas, converts the Date field to datetime, filters for Category = Electricity generation and Unit = GWh, pivots records using date, state, and variable fields, aggregates Total Generation by date, and sorts the observations chronologically.

Expected output: A time-series dataset with dates and aggregated electricity-generation values.

Tests: Check valid dates, chronological order, expected columns, missing values, and duplicate timestamps.

Increment 2: Time-Series Dataset Construction

Objective: Convert the prepared series into supervised learning examples. A custom PyTorch Dataset uses a lookback window for historical observations and a forecast horizon for future target values, returning input and target sequences as tensors.

Expected output: Input-target tensor pairs ready for model training.

Tests: Validate tensor dimensions, sequence length, target alignment, and dataset length.

Increment 3: Transformer Model Development

Objective: Learn temporal dependencies in electricity generation. The model uses a linear input projection, sinusoidal positional encoding, a Transformer encoder with multi-head attention, and an output projection for forecast values. The notebook configuration includes d_model = 64, nhead = 4, three encoder layers, and dropout = 0.1.

Expected output: A trained model that maps historical observations to future generation estimates.

Tests: Confirm successful initialization, expected input/output shapes, and finite predictions.

Increment 4: Multi-Frequency Forecasting

Objective: Run the pipeline at multiple time resolutions. The notebook includes monthly, daily, and hourly data-generation and forecasting workflows. Where the mock-data workflow is used, generated data are synthetic and must not be treated as real operational measurements.

Frequency	Data period in notebook	Purpose
Monthly	January 2019–December 2025	Monthly generation forecast
Daily	January 2024–June 2026	Daily generation forecast
Hourly	April–June 2026	Hourly generation forecast

Tests: Verify timestamp intervals, forecast horizon length, and consistency of output columns.

Increment 5: Testing and Evaluation

- **Unit testing:** Validate preprocessing functions, dataset indexing, and model output dimensions.
- **Integration testing:** Verify the path from data loading through forecast generation.
- **Functional testing:** Run each monthly, daily, and hourly workflow.
- **Data validation:** Check missing values, duplicate timestamps, invalid dates, and non-finite values.
- **Reproducibility:** Repeat runs using the same random seeds.
- **Model evaluation:** Compare forecasts with held-out actual observations using MAE, RMSE, and suitable MAPE calculations.

Forecast outputs alone do not establish predictive accuracy. A chronological train/validation/test split and evaluation against held-out actual values should be implemented and documented before making accuracy claims.

5. Requirements Analysis

Functional Requirements

- The system shall load electricity data from CSV files.
- The system shall preprocess and aggregate generation records.
- The system shall construct time-series input and target sequences.
- The system shall train a Transformer forecasting model.
- The system shall support monthly, daily, and hourly workflows.
- The system shall generate timestamped forecasts.

Non-Functional Requirements

- Reproducible execution using fixed random seeds.
- Reliable handling of time-series records.
- Modular and maintainable Python code.
- Acceptable training and inference time for the selected dataset size.

• Numerically stable training and consistent forecast output formats.

6. System Architecture

Layer	Responsibility
1. Input layer	Historical electricity CSV files or synthetic data.
2. Preprocessing layer	Filtering, date conversion, pivoting, aggregation, and sorting.
3. Dataset layer	Lookback windows and forecast target sequences.
4. Model layer	Input projection, positional encoding, Transformer encoder, output projection.
5. Output layer	Forecast timestamps and predicted generation values in GWh.

7. Technologies Used

Technology	Purpose
Python	Core programming language.
Pandas	CSV loading, data transformation, and aggregation.
NumPy	Numerical operations and synthetic data generation.
PyTorch	Transformer model, tensor processing, and training.
Scikit-learn	Scaling utilities and potential evaluation support.
Matplotlib	Forecast and performance visualization.

8. Deliverables and Completion Criteria

Increment	Deliverable	Completion criterion
1	Preprocessing module	Clean chronological dataset.
2	Time-series dataset	Correctly shaped input-target tensors.
3	Transformer model	Successful training and inference.
4	Forecast pipeline	Timestamped forecast outputs.
5	Testing and evaluation	Verified functionality and measured errors.

9. Advantages of the Incremental Model

- Development is divided into manageable stages.
- Each stage can be tested before extending the system.
- Defects can be isolated to individual components.
- The pipeline can be extended without redesigning the entire system.
- New capabilities can be added as later increments.

10. Limitations and Future Enhancements

- **Synthetic data:** Results from mock data do not establish real-world forecasting accuracy.
- **Project scope:** The implemented target is generation in GWh, not an individual consumer bill in rupees.
- **Validation:** Add a chronological train-validation-test split and report error metrics.
- **Scaling:** Verify scaling is fitted only on training data and predictions are inverse-transformed correctly.
- **Aggregation:** Confirm whether the mean or sum of state-level generation matches the intended national target.
- **Production readiness:** Add model persistence, logging, robust missing-data handling, and a user-facing interface as appropriate.
- **Bill prediction extension:** Add consumer-level consumption data, tariff slabs, fixed charges, taxes, and applicable billing rules if bill calculation is the actual objective.

11. Conclusion

The Incremental SDLC model is suitable for this project because it separates data preparation, time-series construction, model development, forecasting, and evaluation into testable stages. The notebook demonstrates a Transformer-based electricity-generation forecasting workflow. The next priority should be validation using authentic electricity data and held-out observations. If the intended deliverable is consumer electricity-bill prediction, the scope must be extended to consumption and tariff-based billing calculations.

Appendix: Suggested Acceptance Checklist

- Input data schema and units are documented.
- Date parsing, missing values, duplicates, and aggregation rules are checked.
- Training, validation, and test periods are chronologically separated.
- No future observations leak into training inputs.
- Model output shape matches the requested forecast horizon.
- Predictions are evaluated against held-out actual observations.
- Forecast units and limitations are clearly stated.
- Synthetic and real-data results are labeled separately.